



Saint Columban College  
College of Computing Studies  
Pagadian City



## Local Authority

Local Authority 



Easy Web Exploitation picoCTF 2022 inspector

AUTHOR: LT 'SYREAL' JONES

### Description

Can you get the flag?

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is:

NOT\_RUNNING

Launch Instance

### Hints ?

1

How is the password checked on this website?

81,692 users solved



95%



Liked

 picoCTF{FLAG}

Submit  
Flag

**Author:** The Analyst: Hyposelenia

**Challenge:** Local Authority

**Category:** Web Exploitation

**Date:** 02/21/26



## I. Objective

The objective of this challenge is to **understand how client-side authentication works**, identify why it is insecure, and exploit it to gain unauthorized access by inspecting files that are meant to be hidden or protected.

## II. Background

Websites often require users to log in using a **username and password**. A secure website checks these credentials **on the server**, where users cannot see or modify the logic.

However, some insecure websites perform credential checking on the **client side** using JavaScript files. Since JavaScript runs in the user's browser, **anyone can view and read it using Inspect Developer Tools**.

This challenge demonstrates why **client-side authentication is dangerous** and should never be trusted for access control.

## III. Tool Used

- **Web Browser** - Used to access and interact with the challenge website
- **Browser Developer Tools (Inspect)** - Used to view page structure and cookies

## IV. Methodology

1. Launched the challenge instance



Saint Columban College  
College of Computing Studies  
Pagadian City



Its current status is:

NOT\_RUNNING

Launch Instance

2. Accessed the provided challenge website link

Secure Customer Portal

Only letters and numbers allowed for username and password.

Username

Password

Login

3. Observed the landing page and inspected the HTML and CSS structure using Developer Tools. (Right Click > Inspect > Source)

Secure Customer Portal

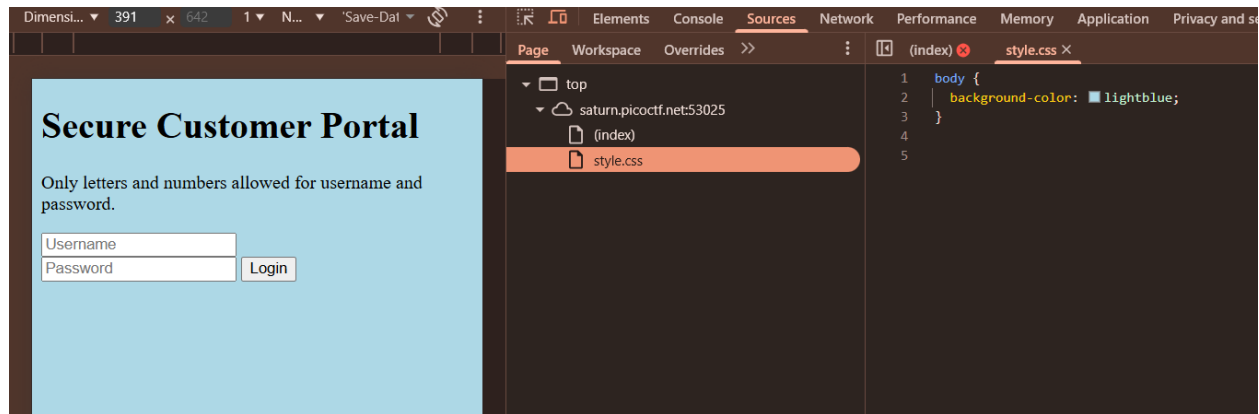
Only letters and numbers allowed for username and password.

input 169.6 x 21.2

Password

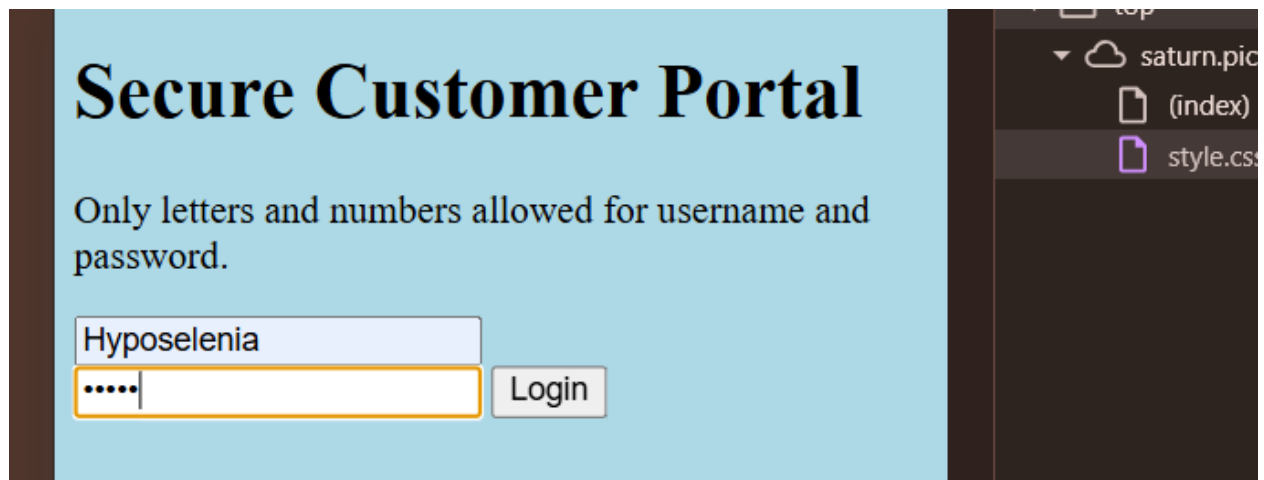
Login

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <link rel="stylesheet" href="style.css">
    <title>Secure Customer Portal</title>
  </head>
  <body>
    <h1>Secure Customer Portal</h1>
    <p>Only letters and numbers allowed for username and password.</p>
    <form role="form" action="login.php" method="post">
      <input type="text" name="username" placeholder="Username" required autofocus>
      <br>
      <input type="password" name="password" placeholder="Password" required>
      <button type="submit" name="login">Login</button>
    </form>
  </body>
</html>
```



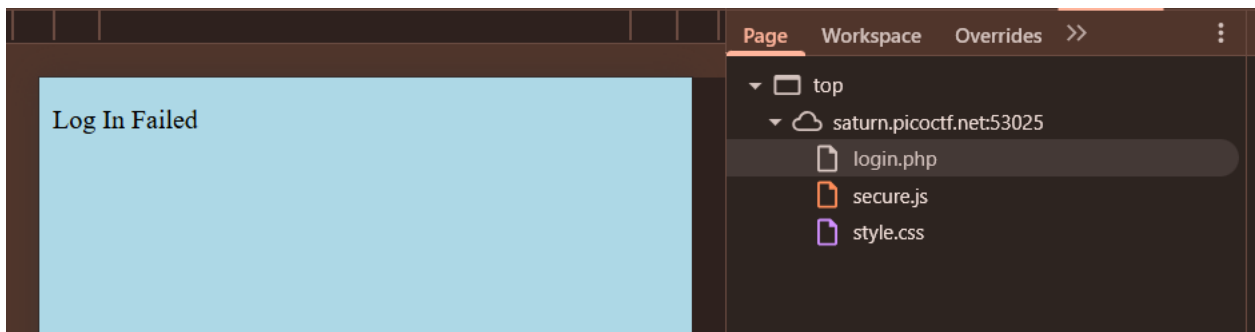
Source Tab is much easier to navigate than element for me ^-^

4. Attempted to log in using random credentials

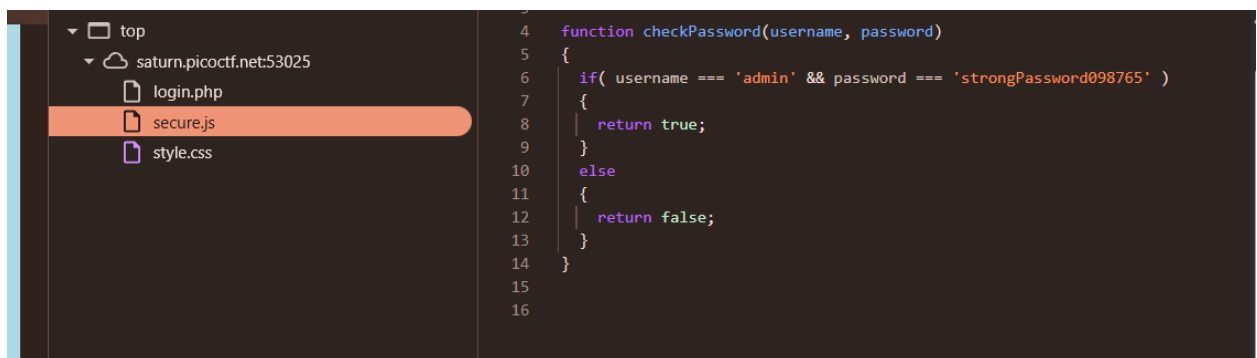


5. After the login failed, inspected the files loaded by the page and noticed:

- login.php handling the login request
- secure.js, a JavaScript file responsible for checking credentials

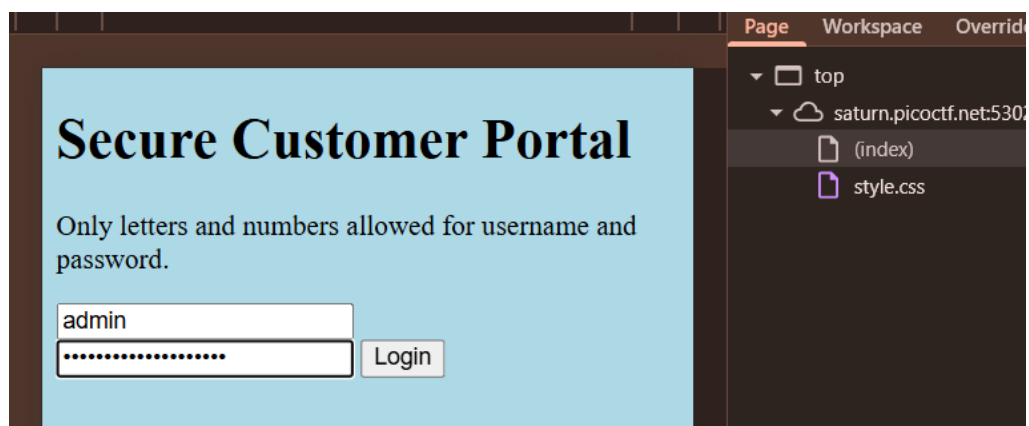


6. Opened secure.js and observed that the **username and password were hardcoded** inside the JavaScript file



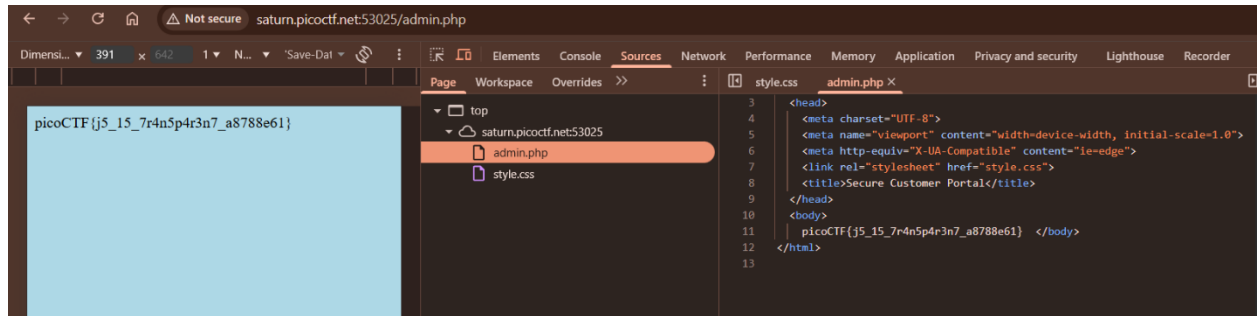
7. Returned to the login page and entered the discovered credentials:

- **Username:** admin
- **Password:** strongPassword098765





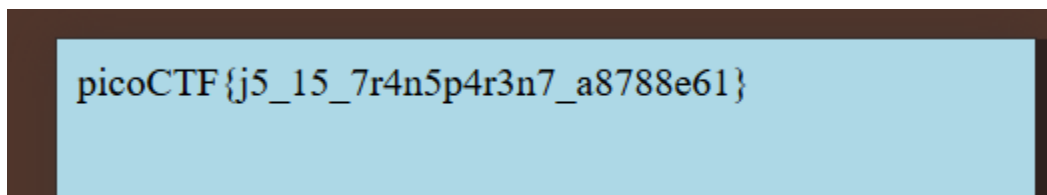
8. Successfully logged in and accessed the admin page



9. The flag was displayed in plain sight and retrieved

## V. Result

**picoCTF{j5\_15\_7r4n5p4r3n7\_a8788e61}**



## VI. Explanation

### Why did the login still work after a failed attempt?

Even though the login failed initially, the website **still sent all the files to the browser**, including **secure.js**. This is because **client-side files are always delivered to the user**, regardless of whether login succeeds or fails.



The browser must download JavaScript files in order to run them — and once downloaded, **anyone can read them**.

Another thing to note, even if you entered the wrong credentials, once a website who was built to cater both successful and failed attempt, there should be a guaranteed display. Login.php, is responsible for displaying the result, taking your credentials and comparing it to a separate file, a function which is connected to the login.php. Gets?

Lovely.

### What is secure.js for and why is it dangerous?

secure.js is a created and named javascript file which contained something like:

- A function that checks:
  - If username equals "admin"
  - If password equals "strongPassword098765"

This means:

- The website trusted **JavaScript** to protect the login
- Anyone using Inspect could see the credentials
- The login system had **no real security**

This is equivalent to:

Locking your house, but taping the key to the front door.

### Why did admin.php load after login?

The admin.php file is just a page meant for administrators. It **does not automatically protect itself** unless server-side checks exist.

Even though it's named .php, it can still behave like plain HTML if:

- No server-side authentication is enforced



- No session validation is performed

Once the client-side check passed, the site allowed access without verifying it securely on the server.

So, **how attackers abuse this in real life?** You might ask.

Attackers:

- Inspect JavaScript files
- Extract credentials, logic, or access rules
- Bypass authentication without guessing passwords

This leads to:

- Admin account takeovers
- Data leaks
- Full website compromise

**How do we prevent this risk?**

- 1. Do authentication on the server, not in JavaScript**
- 2. Never store credentials in client-side files**
- 3. Use sessions and server-side checks** for every protected page
- 4. Restrict access to admin pages on the backend**

Rule of thumb for today:

***If the browser can see it, an attacker can too.***





## VII. Conclusion

The Local Authority challenge highlights a critical security flaw: **trusting the client side for authentication.**

This challenge teaches that:

- JavaScript is transparent and readable
- Client-side checks can always be bypassed
- Real security must be enforced on the server

By completing this challenge, I gained a deeper understanding of why authentication logic must never rely on client-side code and how attackers exploit such weaknesses in real-world applications.

— **The Analyst: Hyposelenia**